# Page 1 - The project structure

## The folders, and why each one exists

```
phishing_twin/
|-- app.py  main.py  live.py  evaluate.py    THE 4 PROGRAMS YOU RUN
|-- README.md  pyproject.toml                config
|-- frontend/     how things LOOK            2 files
|-- backend/      how things WORK           18 files
|-- database/     how things are STORED      3 files
|-- scripts/      run once in a while        3 files
|-- tests/        check a piece works        4 files
|-- data/         the emails, people, answers
+-- docs/         documents about the project (no code uses these)
```

**Why split like this?** One rule: **no file in `backend/`, `database/` or `scripts/` is allowed to use Streamlit.** Streamlit is the tool that makes the website. So the backend does not even know a website exists. That is why the same simulation also runs in the terminal with `main.py` or `live.py`.

## Which file belongs to what

| Group | Files | What it means |
|---|---|---|
| **Frontend** (what you see) | `app.py`, `frontend/theme.py`, `frontend/responsive.py` | The web page, the colours, the phone layout |
| **Backend** (what decides) | the 18 files in `backend/` | The simulation, the model, the email builder |
| **Database** (what is stored) | `database/schema.sql`, `database/build.py`, `database/db.py` | The tables, the filling, the questions |
| **API** | **none** | My frontend and backend are one program, so they call each other directly. There are no web addresses |
| **Configuration** | `pyproject.toml`, `.gitignore` | Makes the folders importable; keeps junk out of Git |
| **AI / Machine learning** | `backend/ml_model.py`, `backend/benchmark.py` | scikit-learn: trains models to compare against my simple one |
| **Data** | `data/hospital_people.json`, `data/hospital_emails_mixed.json`, `data/history_log.jsonl`, `data/simulation.db` | 25 people, 51 emails, every answer, the database |

**Technical words explained: Frontend** = what you see and click. **Backend** = what decides and calculates. **Database** = organised storage you can ask questions of. **API** = an agreed way for one piece of software to ask another for something. **Streamlit** = a tool that turns Python into a web page.

# Page 2 - The important files, one by one

Presented in the order you should show them, not alphabetical.

| # | File | What it does | If you deleted it |
|---|---|---|---|
| 1 | `app.py` | The whole website: sidebar + 8 tabs. 1,416 lines | No website. The terminal versions still work |
| 2 | `backend/engine.py` | **One email delivery**: pick, guess, answer, learn. The heart | Live and Run tabs stop |
| 3 | `backend/twin.py` | The model itself, and all file saving. 706 lines | Everything stops |
| 4 | `backend/reader.py` | Decides if a person spots a fake email, by looking at it | Nobody can answer an email |
| 5 | `backend/human.py` | The older way: rolls dice against secret numbers | The comparison with the old method is lost |
| 6 | `backend/email_builder.py` | Turns a real email into a fake one | No new emails; only the fixed 51 |
| 7 | `backend/modifications.py` | The 17 ways to make an email look fake | The builder cannot damage anything |
| 8 | `backend/entities.py` | Swaps ticket numbers, wards, days, times for new ones | New emails would be exact copies |
| 9 | `backend/situation.py` | The moment: day or night, calm or busy | People would never be rushed |
| 10 | `backend/feedback.py` | Explains what the person got right or wrong | No explanations shown |
| 11 | `backend/metrics.py` | Turns the history into tables for the charts | The dashboard cannot draw |
| 12 | `backend/store.py` | Saves the app's own files | Built emails and human answers are not kept |
| 13 | `backend/ml_model.py` | The machine learning: 15 features, 4 models | The Model tab stops |
| 14 | `database/schema.sql` | Describes the 5 tables, in plain SQL | The database cannot be built |
| 15 | `database/build.py` | Fills the database from the data files | Same |
| 16 | `database/db.py` | The questions you can ask the database | **No emails load. The app will not start** |
| 17 | `frontend/theme.py` | The colours and the result words | Charts lose their colours |
| 18 | `data/history_log.jsonl` | **Every answer ever given.** The real truth | All learning is lost forever |

### The three most important things to understand

**1.** `backend/twin.py` **holds the entire model, and it is tiny.** The whole prediction is one division:

```
P(click) = yes / (yes + no)        yes = times clicked, no = times reported
```

**2.** `backend/engine.py` **does one delivery in four fixed steps** - pick an email, guess, let the person answer, learn. The guess happens **before** the answer exists.

**3.** `data/history_log.jsonl` **is the truth.** Every answer is added to the end and never changed. The database and the summary files are all rebuilt from it, so they can be deleted safely.

# Page 3 - How everything runs, start to finish

### When you launch the project

```
YOU type:  python3 -m streamlit run app.py
    |
    v
app.py          loads 25 people      -> backend/persona.py
                loads 51 emails      -> backend/generator.py -> database/db.py
                rebuilds the model   -> backend/twin.py reads history_log.jsonl
                draws the 8 tabs     -> frontend/theme.py + responsive.py
    |
    v
YOU click "Start" on the Live tab
    |
    v
app.py  live_step()  ->  backend/engine.py  deliver()
    |
    +-- 1. PICK    backend/email_builder.py  (+ modifications.py, entities.py)
    +-- 2. MOMENT  backend/situation.py
    +-- 3. GUESS   backend/twin.py     predict()      <- BEFORE the answer
    +-- 4. ANSWER  backend/reader.py   respond()
    +-- 5. LEARN   backend/twin.py     update()  -> data/history_log.jsonl
    +-- 6. EXPLAIN backend/feedback.py
    |
    v
app.py draws the email, the colour, the counters and the chart
```

### The key code, in `backend/engine.py` lines 112-116

```
situation = make_random_situation()                              # the moment
predicted = self.twin.predict(person.id, email)                  # THE GUESS
action = self.answerer_for(person.id).respond(email, situation)  # THE ANSWER
self.twin.update(person.id, email, action, source="sim")         # LEARN
```

**Why the order matters:** if you swapped the guess and the answer, the program would still run and report excellent results, and every result would be worthless - the twin would be copying the answer instead of predicting it.

### How the data moves

```
hospital_people.json  -> persona.py  -> Persona objects (25)
hospital_emails_mixed.json -> database/build.py -> simulation.db
                                                -> database/db.py -> generator.py
every answer -> twin.py update() -> history_log.jsonl   (the truth)
                                 -> history.csv, results.csv
history_log.jsonl -> twin.py rebuild_from_log() -> the model, on restart
history_log.jsonl -> ml_model.py -> the machine learning table
```

# Page 4 - Your demonstration plan

## Open the files in this order

| Open | Point at | Say |
|---|---|---|
| **1.** `app.py` line 260 | `st.tabs([...])` | "This is my whole website. Eight tabs, all written in Python - Streamlit turns it into a web page, so I wrote no HTML." |
| **2.** `app.py` line 115 | `answerer_for()` | "This is how my sidebar switch works. It passes a *function*, so the engine always asks who is answering right now." |
| **3.** `backend/engine.py` line 112 | the four lines | "This is the heart of the project. One delivery: pick, guess, answer, learn. The guess is made before the answer exists - that is what makes it a real prediction." |
| **4.** `backend/twin.py` line 312 | `p_click()` | "This is my entire model: clicks divided by clicks plus reports. Seven lines. I later built a 200-tree machine learning model and it only beat this by half a percent." |
| **5.** `backend/reader.py` line 34 | `CHECKS_BY_SKILL` | "This decides whether a person spots a fake. People differ by *which checks they do*, not by luck. Careless does 1 check, careful does all 8." |
| **6.** `backend/reader.py` line 189 | `checks_for()` | "When someone is rushed, they drop to the first 2 checks. Being busy does not change what you know, it changes how much you bother to look." |
| **7.** `database/schema.sql` line 70 | the `hidden_probs` table | "The secret click numbers live in their own table on purpose. The twin must never read them - they are only used at the end to mark its work." |
| **8.** `data/history_log.jsonl` | any line | "Every answer ever given, one per line, added at the end and never changed. If I delete everything else, I can rebuild it all from this." |

## Ten questions, with short answers

**Q: Is this a frontend-backend project?** Yes, but both halves run in one Python program. The split is enforced by a rule: no backend file may use Streamlit.

**Q: What are your API endpoints?** There are none, and I would not invent any. The frontend calls backend functions directly, like `twin.predict()`.

**Q: Why Streamlit?** It gave me a working 8-tab website in Python alone, with no HTML or JavaScript. My audience is a researcher, not a customer.

**Q: Why SQLite?** It is one file, needs no server, and comes free with Python. I have a few thousand rows and one user.

**Q: What is your model?** Clicks divided by clicks plus reports, per person per type of pressure. It is the standard estimate for a yes/no probability.

**Q: Did machine learning do better?** Barely. Random forest 92.0%, my simple model 91.5%, on emails neither had seen. I think that is a real finding.

**Q: Where did the data come from?** I generated it. The 51 emails come from the HIM 1055 course table of attacks, so each names the technique it shows.

**Q: Is fake data a weakness?** Yes, the main one. But it gives me a known correct answer to check against, which real people never would.

**Q: How did you test it?** Not well enough. The 4 files in `tests/` print results for a human to read; they do not check themselves automatically.

**Q: Which parts were AI-assisted?** I built it with Claude Code and kept `docs/prompts.md` as a record. The typing was assisted; the decisions about testing honestly are mine, and I can explain any file.

## If you only remember one thing

Open `backend/engine.py` at line 112 and explain those four lines. Pick, guess, answer, learn - in that order. Everything else in the project exists to serve that loop.